Frontend Developer · GitHub ↗

김수환 포트폴리오

실시간 UI 성능·공통 컴포넌트·멀티 플랫폼 구현 사례

(주)다온소프트랩 · 업무 프로젝트

Bitria

멀티 거래소 연동 암호화폐 거래 플랫폼

2025.11 – 현재 | 프론트엔드·백엔드 개발 담당 · 2인 팀

여러 암호화폐 거래소를 연동해 현물·선물·마진 거래를 제공하는 플랫폼입니다.
2인 팀에서 웹·모바일 프론트엔드와 백엔드 개발을 담당하고, 인프라 운영에 참여했습니다.

담당 범위

- 실시간 구독 분리: 거래 화면 전체 React root 렌더 시간 656.3 → 393.5ms(개발 빌드·5회 중앙값)
- 모바일 홈의 shared/ui 직접 import 적용: TBT 417 → 292.5ms(로컬 프로덕션 빌드·5회 중앙값)
- 주문 폼·레버리지 모달의 최대 주문 가능액 계산과 랏 단위 절사를 공통 모듈로 통합
- 실시간 시세·호가·차트 및 주문 화면 개발, 웹·iOS·Android·Electron 대응과 모바일 앱 출시

실시간 갱신을 표시 컴포넌트 안으로 제한

문제 페이지 최상위의 시세 state가 주문 폼과 하단 패널까지 다시 렌더링했습니다.

판단·수정 시세 표시 컴포넌트 안으로 구독을 옮기고, props와 구독 배열의 참조를 안정화했습니다.

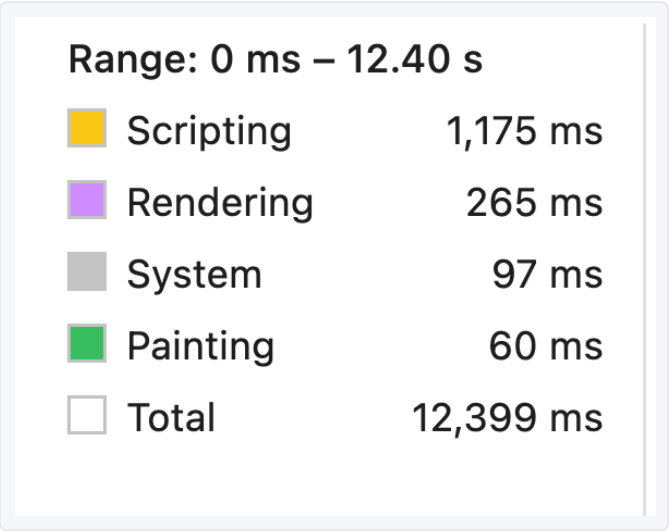
거래 화면 전체 React 트리 · 동일 시세 12초 재생 · 전후 각 5회 중앙값

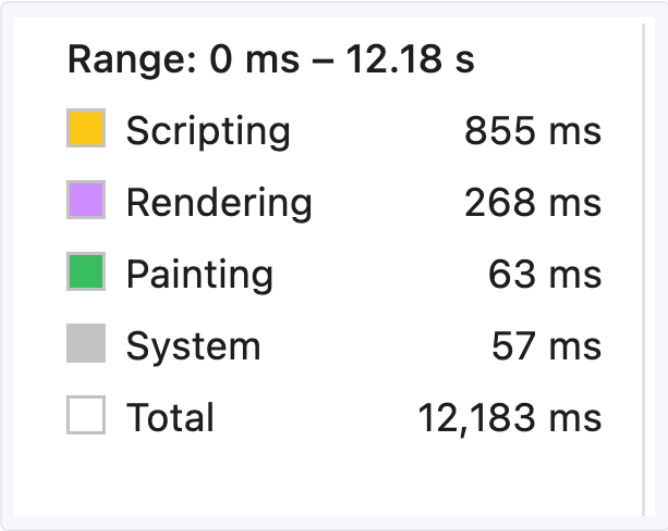
지표	개선 전	개선 후
React root 렌더 시간 합계	656.3ms	**393.5ms**
전체 렌더 작업	5,241회	**1,534회**
주문 폼 / 하단 패널 렌더	각각 8회	**각각 0회**

Chrome 156 · Apple M1 Pro / Metal · 개발 빌드 · 390×844.
같은 시세 메시지 73개를 재생했습니다.
호가·체결·차트 등 마운트된 React 트리를 자체 hook으로 계측했으며, 결과 JSON과 Chrome trace를 보관했습니다.
아래 캡처는 해당 실행의 Chrome Performance 기록입니다.
표의 React 횟수·시간은 다운로드 파일의 자체 hook 기록으로 확인할 수 있습니다.
React DevTools Profiler 캡처가 아닙니다.
기기 부하는 전후 고정하지 못했으므로 시간 감소율은 이번 실험의 관측값입니다.

전후 각 5회 렌더 계측 데이터(JSON) 다운로드 ↗ · 원본 Chrome trace·캡처 ZIP 다운로드 ↗

Chrome Performance 실행 시간 비교





개선 전 · Scripting 1,175ms · run-01의 Summary 확대

전체 기록 보기 ↗

개선 후 · Scripting 855ms · run-02의 Summary 확대

전체 기록 보기 ↗

대표 실행의 Summary 영역을 확대한 원본 캡처입니다.
Scripting은 브라우저의 JavaScript 실행 시간으로, 위 표의 React 렌더 시간과 다른 지표입니다.
Total은 기록 구간 길이입니다.

모바일 홈의 초기 JavaScript 실행 비용 축소

문제 shared/ui 일괄 재수출로 버튼을 사용하는 화면에 불필요한 모듈까지 함께 묶였습니다.

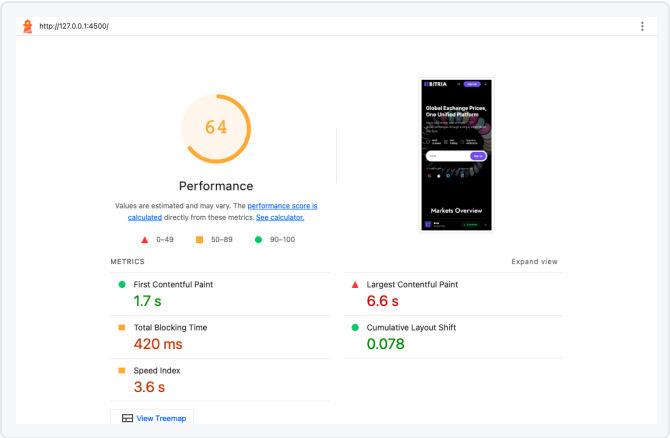
판단·수정 optimizePackageImports 적용은 청크 수와 전체 번들이 늘어 되돌렸습니다.
직접 import와 three.js 배경 지연 로딩을 적용했습니다.

shared/ui 직접 import 변경 전후 · 로컬 프로덕션 빌드 · 5회 중앙값

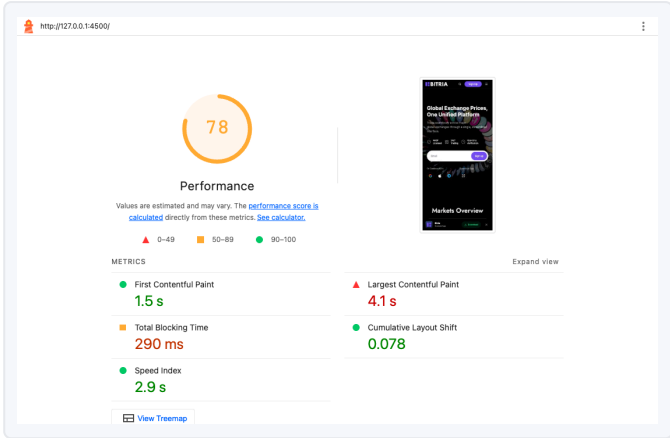
지표	개선 전	개선 후
총 차단 시간(TBT)	417ms	**292.5ms**

Chrome 156 · Apple M1 Pro / Metal · 390×844 · CPU 4배·RTT 150ms·1638.4Kbps.
three.js 지연 로딩은 양쪽에 이미 포함돼 있습니다.
실행별 JSON·HTML·trace·화면을 보관했습니다.
변동 폭을 포함한 전체 결과를 함께 기록했습니다.

Lighthouse 원본 보고서 전후 비교



개선 전 Lighthouse 원본 보고서 · 5회 중 TBT 중앙값 실행 · 화면은 420ms로 반올림 표시



개선 후 Lighthouse 원본 보고서 · 5회 중 TBT 중앙값 실행 · 화면은 290ms로 반올림 표시

최대 주문 가능액의 계산과 절사 규칙 통합

문제 레버리지 모달과 주문 폼이 별도 계산식을 사용해 최대 주문 가능액이 달랐습니다.

판단·수정 수수료·안전 여유분·거래소별 마진 정책을 공통 계산 함수로 모으고, BigNumber 기반 랏 단위 내림을 함께 적용했습니다.

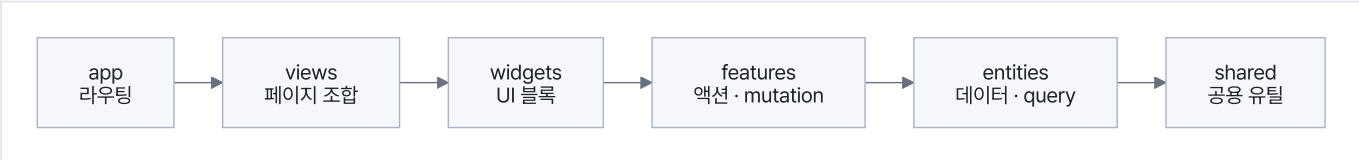
BTC 격리 마진 100·25배와 INJ 교차 마진 50배의 주문 가능 랏 수 일치를 확인했습니다.
금액 표시는 조회 시점의 시세에 따라 달라질 수 있습니다.

구현 구조와 설계

FSD 아키텍처와 화면 분기 기준

화면과 도메인이 늘어나도 의존성이 뒤섞이지 않도록 app → views → widgets → features → entities → shared 순의 단방향 규칙을 정했습니다.
조회(query)는 entity에, 변경(mutation)은 feature에 두어 각 레이어의 책임을 분리했습니다.

웹과 앱의 라우트가 다르면 폴더를 분리했습니다.
화면 크기 차이가 작으면 prop으로 분기하고, 마크업 차이가 크면 디바이스별 파일을 두도록 기준을 정했습니다.

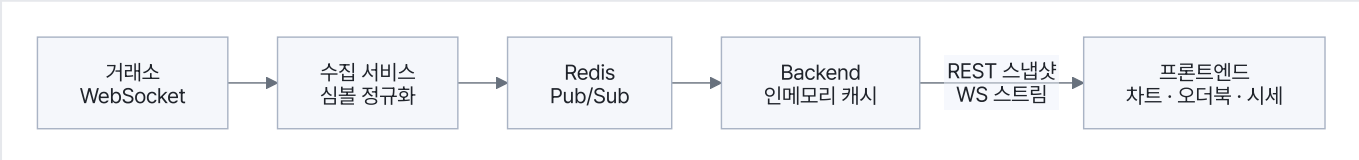


FSD 레이어 간 의존 방향과 각 레이어의 책임

실시간 데이터 전달과 구독 구조

거래소 WebSocket 데이터는 수집 서비스의 심볼 정규화와 Redis Pub/Sub을 거쳐 백엔드에 전달됩니다.
프론트엔드는 REST 스냅샷과 WebSocket 스트림으로 차트·호가·시세를 표시합니다.

실시간 구독을 값을 표시하는 컴포넌트로 옮기고, 티커 데이터도 항목별로 구독하도록 분리했습니다.
보이지 않는 오더북과 닫힌 드로어는 구독을 중지하고, 체결 내역 행에는 memo를 적용해 갱신 범위를 줄였습니다.

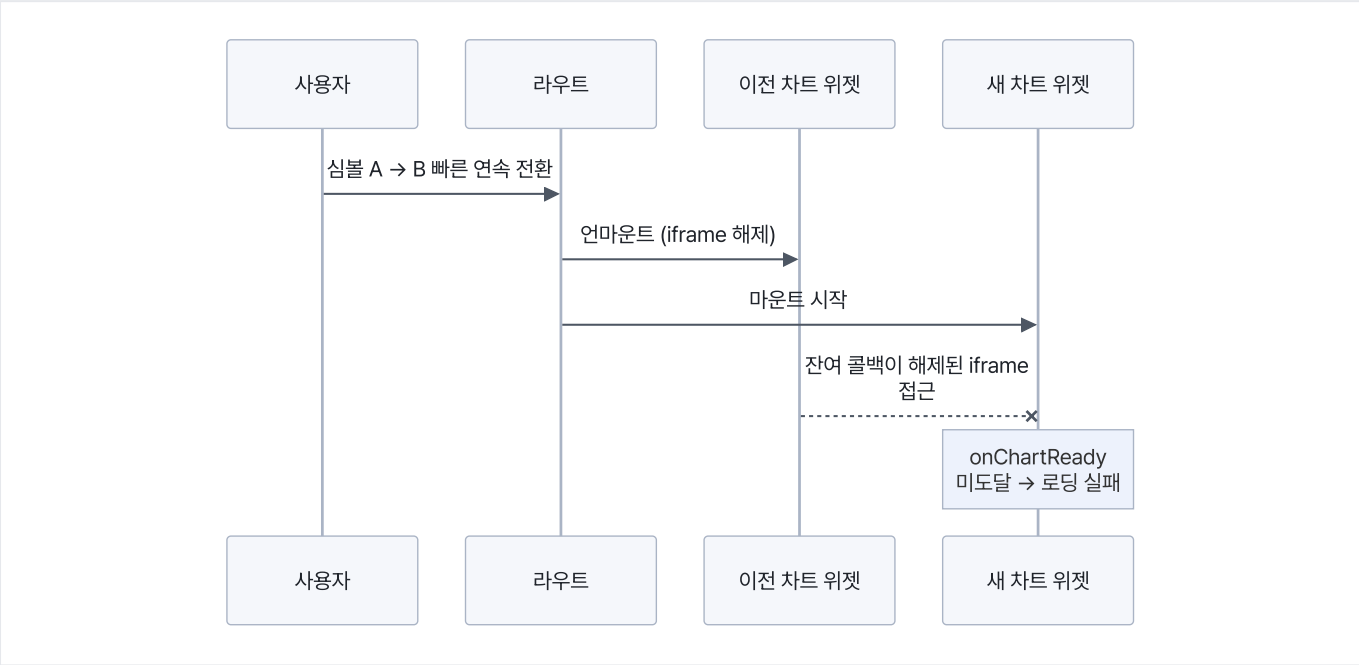


실시간 데이터가 프론트엔드에 전달되는 흐름

차트 로딩 실패의 원인 분석과 해결

간헐적인 차트 로딩 실패를 클라이언트 오류 로그와 서버·CDN 응답 시간으로 추적했습니다.
심볼을 빠르게 바꾸면 이전 위젯의 콜백이 이미 해제된 iframe에 접근해 새 차트가 초기화되지 않는 상황을 재현했습니다.

라우트가 바뀔 때 페이지 전체가 다시 마운트되는 구조가 원인이었습니다.
차트 뷰를 레이아웃에 두어 불필요한 재마운트를 막고, 위젯 초기화 중 심볼이 바뀌어도 최신 심볼을 적용하도록 수정했습니다.



수정 전: 빠른 심볼 전환에서 발생한 위젯 생명주기 충돌

> **수정 후**
>
> 1. 거래 레이아웃에서 차트 위젯 유지
> 2. 심볼 전환 시 최신 심볼을 ref에 저장
> 3. onChartReady에서 최신 심볼 반영
> 4. 준비 완료 후 전환은 setSymbol로 처리

주문 API와 비동기 처리 서비스

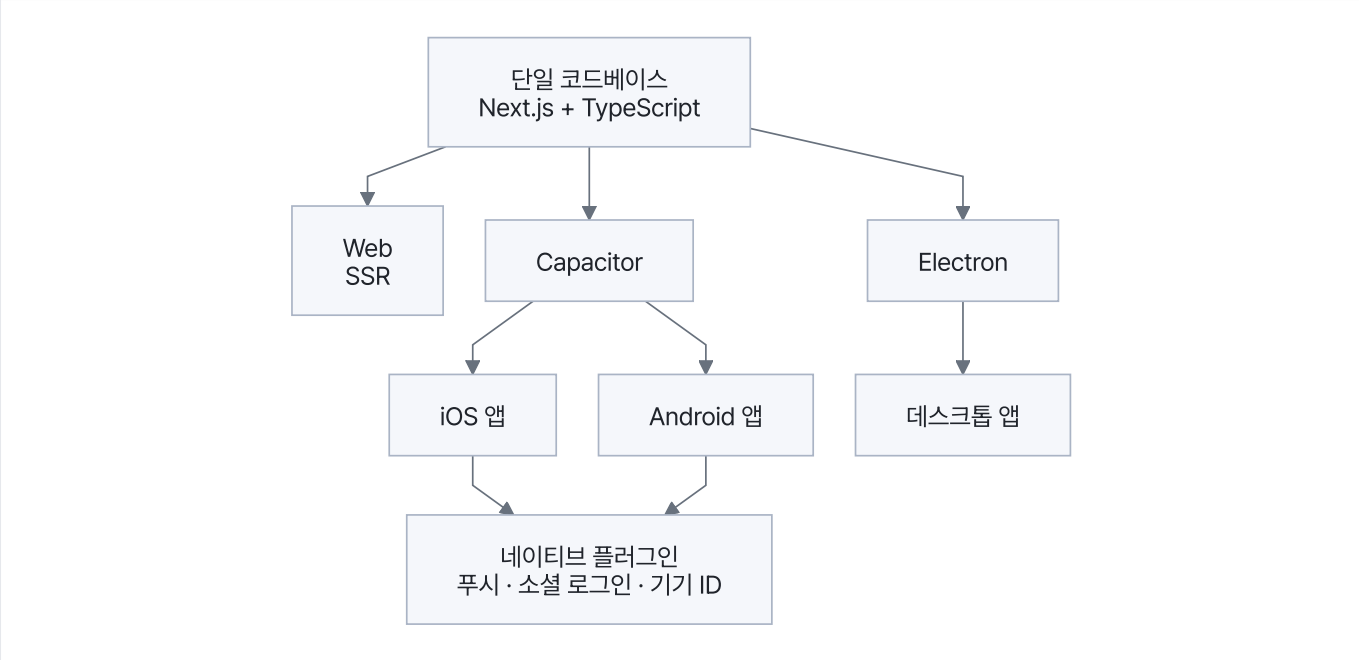
2인 팀에서 프론트엔드와 백엔드 개발을 담당했습니다.
주문·인증·자산 API와 함께 주문 요청을 검증하고 처리 서비스로 전달하는 흐름을 구현했습니다.

주문 요청은 receiver → processor → executor로 전달됩니다.
각 프로세스는 RabbitMQ 큐로 연결하고, 주문 접수·처리·실행 단계를 분리했습니다.
화면의 주문 상태와 백엔드 처리 흐름을 함께 다뤘습니다.

단일 코드베이스의 멀티 플랫폼 대응

Next.js·TypeScript 코드베이스를 웹, Capacitor 기반 iOS·Android 앱, Electron 데스크톱 앱으로 빌드했습니다.
모바일 앱을 스토어에 출시하고 푸시(APNs·FCM)·소셜 로그인·기기 식별 기능을 연결했습니다.

앱 강제 업데이트 정책을 연동하고, Electron 앱의 API 요청을 식별하는 헤더를 추가했습니다.



웹·모바일·데스크톱 빌드와 네이티브 기능 연결 구조

Bitria 상세 설명 ↗

(주)다온소프트랩 · 업무 프로젝트

Gobit

코인 정보·커뮤니티 플랫폼

2024.10 – 2025.12 | 프론트엔드·백엔드 개발 담당 · 2인 팀

주요 개발 기간 · 이후 유지보수, 2026.07–08 인프라 이전

암호화폐 시세·차트·뉴스·커뮤니티와 모의투자 기능을 제공하는 코인 정보 플랫폼입니다.
2인 팀에서 Next.js 웹 클라이언트를 새로 구축하고 Node.js·Express API를 개발했습니다.
Binance 시세를 수집해 Redis에 캐시하고 Socket.IO로 전달했으며, 회원·게시글 기능과 운영 도구를 구현했습니다.

담당 범위

- 시세·차트·뉴스·커뮤니티 화면 구현 및 TradingView 차트 연동
- Binance WebSocket·Redis·Socket.IO 기반 실시간 시세 처리
- 커뮤니티 기능과 회원 인증·데이터 API 개발

분산된 레거시 서비스의 배포 구조 통합

문제 여러 서버에 분산된 서비스와 소스 없이 남은 시세 수집 바이너리의 유지보수가 필요했습니다.

판단·수정 수집기를 Node.js로 재작성하고, 수집기 → Redis → 서버로 데이터 흐름을 정리했습니다.
서비스를 모노레포와 Kubernetes 환경으로 이전하고 빌드·배포를 자동화했습니다.

2026.07–08 인프라 이전 작업입니다.
저장소에서 수집기·서비스별 Dockerfile·Kubernetes 매니페스트·GitHub Actions 구성을 확인할 수 있습니다.

Gobit 상세 설명 ↗

두잇 · 업무 프로젝트

doit-uikit

사내 공통 UI 라이브러리

2024.02 – 2024.07 | 프론트엔드 개발 담당

프로젝트마다 반복 구현하던 UI를 공통 라이브러리로 만들고, React 개발 규칙을 팀에 공유했습니다.

담당 범위

- Atomic Design 기반 UI 컴포넌트 24종 개발 및 npm 배포
- 타입 선언 자동 생성·React 외부화 등 패키징 구성
- Storybook·Chromatic 문서화 및 사내 프로젝트 2곳에 도입

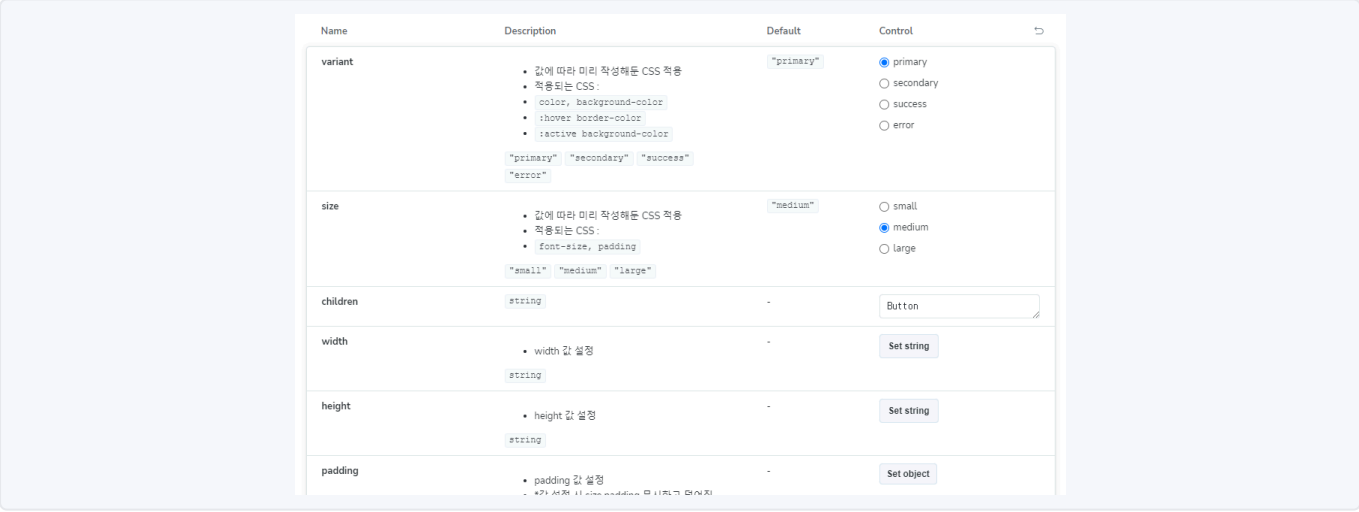
구현 구조와 설계

컴포넌트 조합과 라이브러리 배포

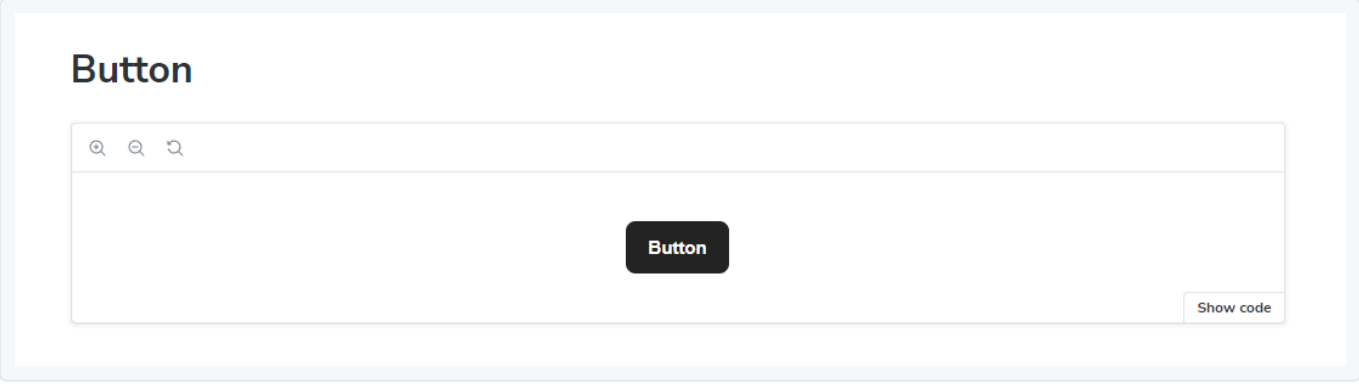
Button·Input 같은 Atom을 조합해 FileInput 등의 Molecule을 구성했습니다.
Storybook에서 Props·상태·사용 예를 확인하고 Playground로 직접 값을 바꿔볼 수 있도록 문서화했습니다.

vite-plugin-dts로 타입 선언을 함께 배포하고 React·React DOM은 peer dependency로 분리했습니다.
D.A.X와 회사 홈페이지에 적용하면서 발견한 요구사항을 라이브러리에 반영했습니다.

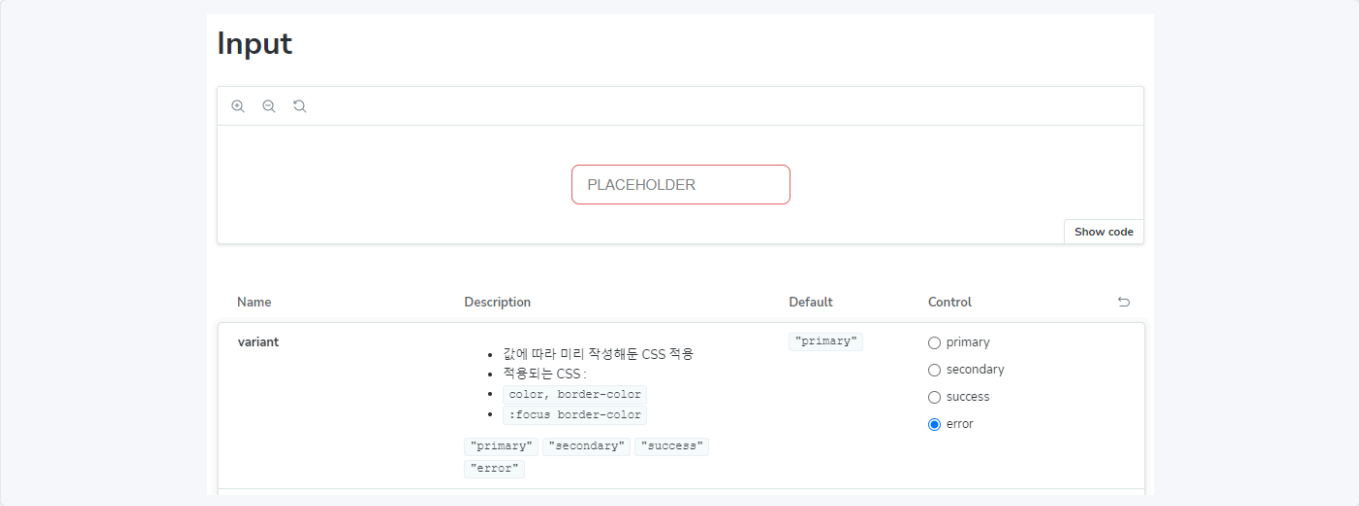
doit-uikit · 프로젝트 화면



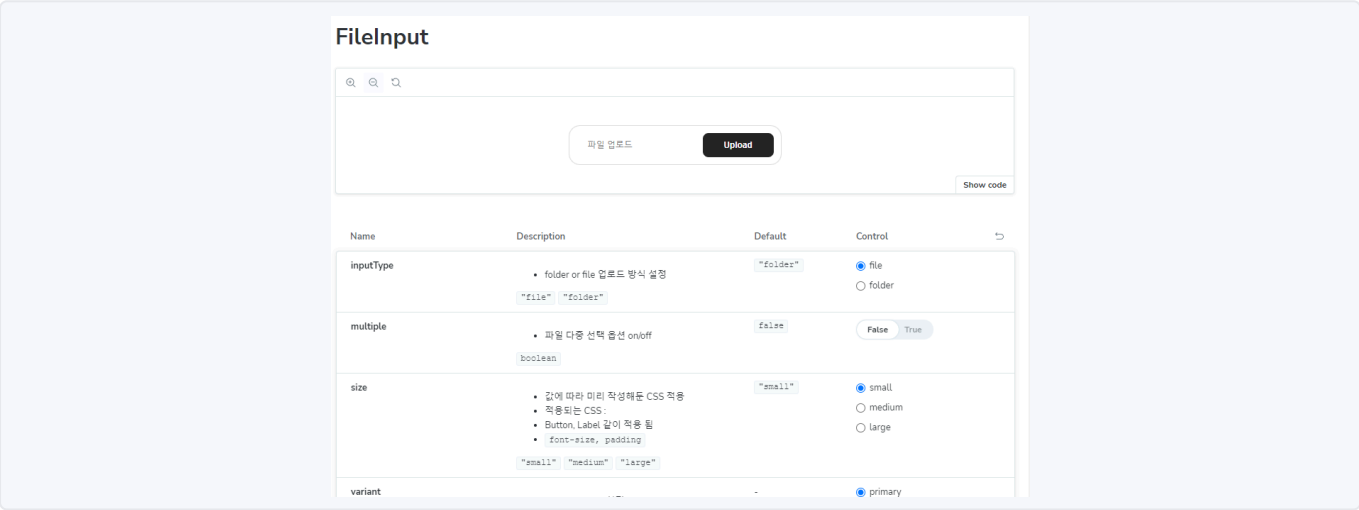
doit-uikit 컴포넌트의 Storybook 문서와 Props 설정 화면



Button 컴포넌트의 Storybook 예시



Input 컴포넌트의 상태·Props 문서



Input·Button·Label을 조합한 FileInput 컴포넌트

doit-uikit 상세 설명 ↗

두잇 · 업무 프로젝트

D.A.X

AI 모델 학습·운영 플랫폼

2024.01 – 2024.06 | 프론트엔드 개발 담당 · 백엔드 1명과 협업

데이터 수집·라벨링부터 모델 학습·추론까지 지원하는 사내 플랫폼입니다.
백엔드 개발자 1명과 협업해 프론트엔드 전 화면을 개발했습니다.

담당 범위

- 데이터 관리·모델 학습·추론·대시보드 화면 구현
- 원본↔화면 좌표 변환을 적용한 캔버스 라벨링 도구 개발
- 이미지 업로드·컬렉션 ZIP 다운로드 구현 및 사내 UI 라이브러리 적용

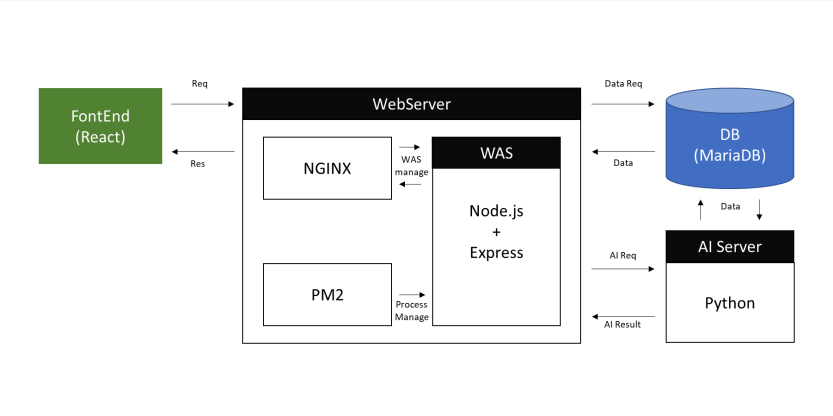
> 라벨링 이미지는 화면 크기에 맞춰 축소되지만 학습용 좌표는 원본 해상도로 저장해야 했습니다.
> 원본↔화면 좌표의 양방향 변환을 전용 훅으로 분리해 그리기·저장·재표시가 같은 규칙을 사용하도록 구현했습니다.

구현 구조와 설계

서비스 구성과 프론트엔드 담당 범위

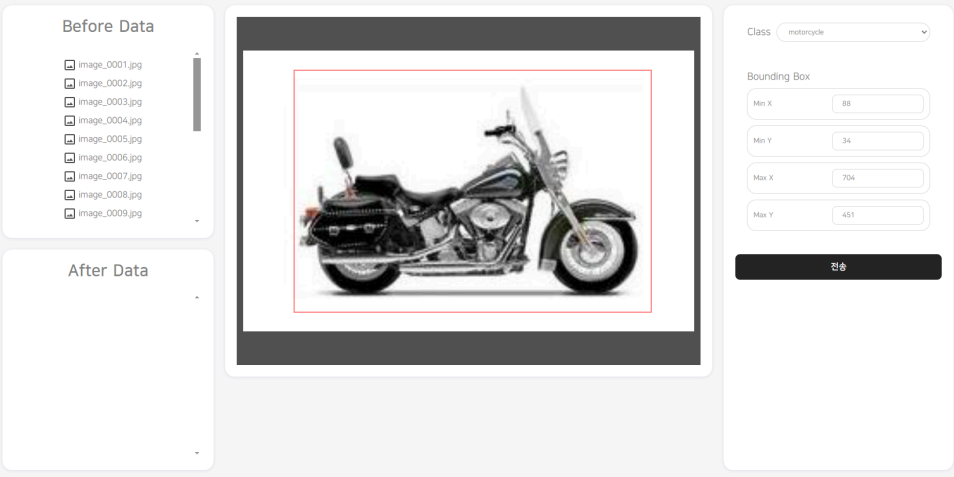
React 프론트엔드는 웹 서버 API를 통해 프로젝트·라벨·모델 데이터를 조회하고 변경합니다.
백엔드와 AI 서버가 연결된 전체 구성에서 프론트엔드 화면과 API 연동을 담당했습니다.

사내 표준 디렉토리 구조를 적용하고 API 호출을 auth·project·class·datamanage·aiModel·operation 등 도메인별 서비스 모듈로 나눴습니다.
원본·렌더 좌표 변환은 useCoordinateTransform 훅으로 분리해 그리기·저장·재표시에 같은 규칙을 적용했습니다.

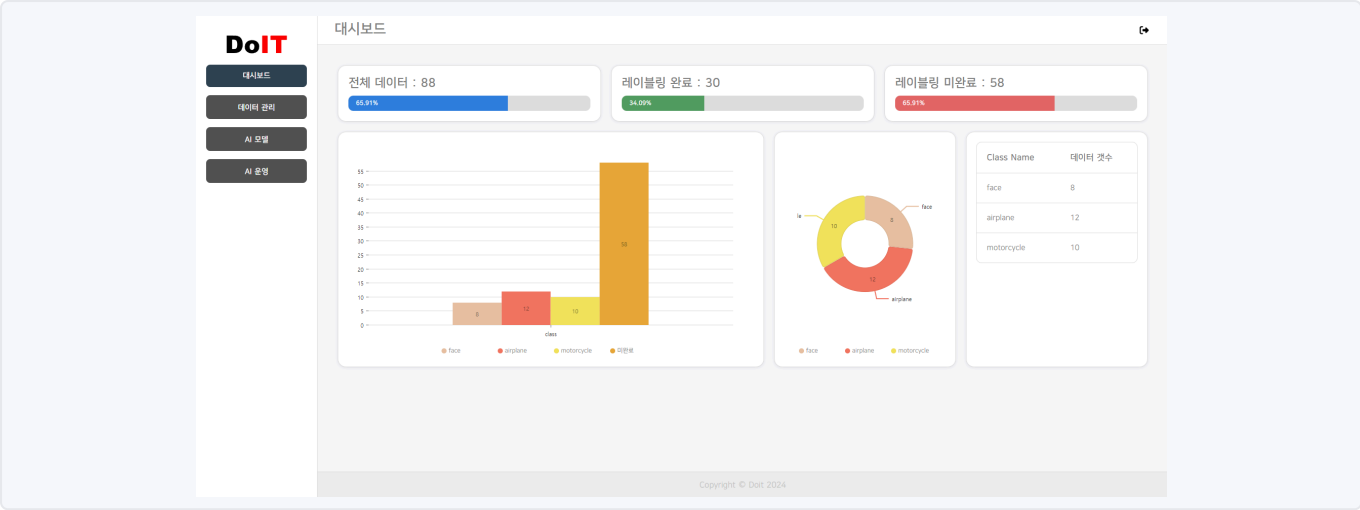


D.A.X 전체 서비스 구성 — 담당 범위는 React 프론트엔드와 API 연동

D.A.X · 프로젝트 화면



D.A.X에서 이미지 위에 바운딩 박스를 그려 라벨링하는 화면



D.A.X의 프로젝트 데이터와 학습 현황 대시보드

D.A.X 상세 설명 ↗

두잇 · 업무 프로젝트

doit-homepage

회사 홈페이지 React 마이그레이션

2024.05 – 2024.07 | 프론트엔드 개발 담당

jQuery 기반 회사 홈페이지를 React·TypeScript·Vite SPA로 재작성해 한국어·영어와 반응형 화면을 지원했습니다.

담당 범위

- 주요 5개 페이지를 React로 재작성하고 공통 UI를 doit-uikit으로 통일
- i18next 기반 한국어·영어 및 반응형 화면 구현

doit-homepage 상세 설명 ↗

두잇 · 업무 프로젝트

XRGIS BackOffice

XR·GIS 원격 협업 시스템 백오피스

2023.08 – 2023.12 | 프론트엔드·백엔드 개발 담당

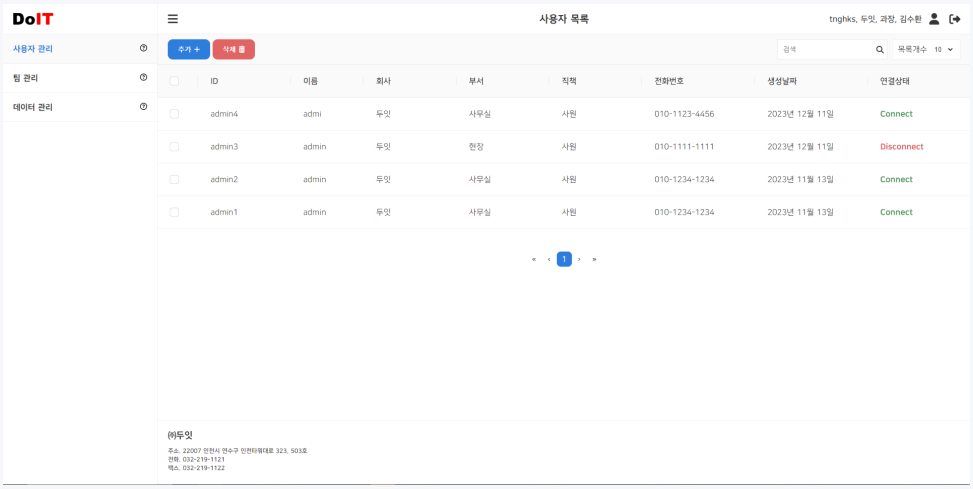
가상·증강·혼합현실(XR)과 지도·공간정보(GIS)를 결합한 원격 협업 서비스입니다.
사용자·팀 관리와 OBJ 모델·영상 파일 처리를 위한 백오피스를 개발했습니다.
React 화면과 Express API를 구현하고 GridFS 기반 파일 저장 및 공통 세션 인증을 연동했습니다.

담당 범위

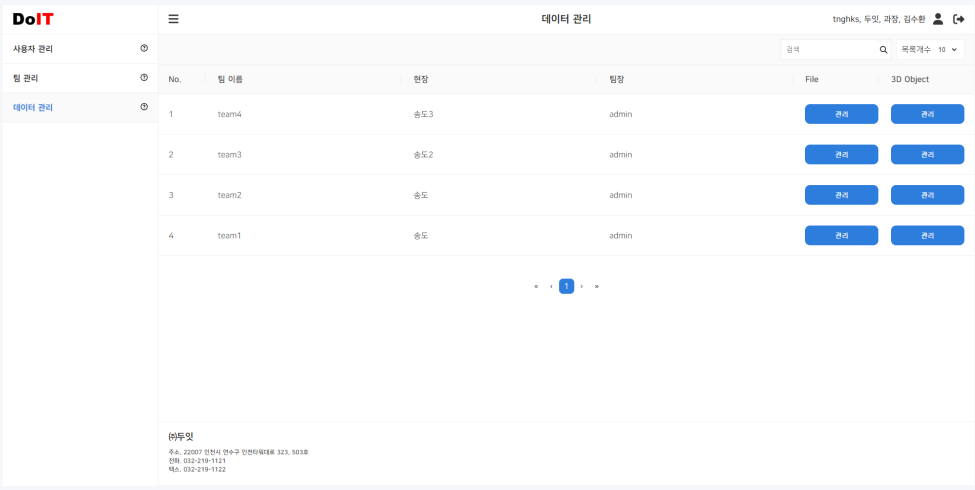
- 사용자·팀·파일 관리 화면 및 Express API 개발
- GridFS 대용량 파일 업로드와 ZIP 일괄 다운로드 구현
- 사내 공통 계정 인증 및 팀별 파일 접근 권한 적용

> MongoDB에 파일을 저장해야 하는 요구사항에 맞춰, 일반 문서의 16MB 제한에 걸리는 OBJ·영상 등을 GridFS의 청크 분할 저장으로 처리했습니다.
> 같은 관리자 계정을 사용하는 사내 서비스와는 express-mysql-session 기반 세션 저장소를 통해 인증 체계를 맞췄습니다.

XRGIS BackOffice · 프로젝트 화면



XRGIS 사용자 관리 화면



XRGIS 팀별 데이터·파일 관리 화면

XRGIS BackOffice 상세 설명 ↗